

SESSION 04

Hyperparameter Optimization & Experiment Tracking

Automatically tune model settings with Optuna and track every experiment with MLflow.

Day 1 | 2:00 PM – 3:00 PM | 60 Minutes

Session Goals & Objectives



Core Purpose

Learn to automatically find the best model settings using intelligent search strategies — moving beyond manual guessing to systematic optimization.

Pair optimization with MLflow experiment tracking so every trial, metric, and artifact is logged and reproducible.



Key Learning Outcomes

By the end of this 60-minute session, you will be able to:

- ✓ Distinguish hyperparameters from model parameters
- ✓ Implement Bayesian Optimization with Optuna
- ✓ Track experiments and artifacts with MLflow
- ✓ Analyse results with parallel coords & importance plots

Hyperparameters vs. Model Parameters



Hyperparameter

YOU set it BEFORE training starts

`learning_rate`

`max_depth`

`n_estimators`

`min_samples_split`

`regularization strength (C,  $\lambda$ )`

Controls the learning process — tuning these is the goal!



Model Parameter

The ALGORITHM learns these from data

`weights (neural network)`

`split thresholds (trees)`

`biases`

`coefficients (linear models)`

`cluster centroids (k-means)`

You never set these manually — they emerge during training.

Search Strategies Compared



Exhaustive

Grid Search

Tries EVERY combination of values in a predefined grid.

Pros

- Thorough — misses nothing

Cons

- Exponential cost
- Slow with many params



Faster

Random Search

Randomly samples combinations from the search space.

Pros

- Faster than grid search
- Often finds good configs quickly

Cons

- No learning from past tries



Smartest

Bayesian Optimization

Builds a surrogate model and learns which regions to explore next.

Pros

- Learns from every trial
- Best for expensive models

Cons

- Slightly more setup

Rule of thumb: Random search beats grid when only 2-3 hyperparameters matter. Bayesian wins for expensive training runs.

How Optuna Works — Bayesian Optimization

Optuna learns from every trial to propose smarter settings next time

1

Define Objective

Write a function that takes a trial and returns a score (e.g. AUC).

2

Suggest Params

`trial.suggest_int()` / `suggest_float()` pick values each run.

3

Train & Evaluate

Train model, score with cross-validation. Return the score.

4

Optuna Learns

TPE surrogate model guides the next suggestion.

5

Best Result

retrieve `best_params` and `best_value` from the study object.

```
study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=50)      # Runs 50 intelligent trials
print('Best AUC:', study.best_value)       # e.g. 0.9673
print('Best Params:', study.best_params)    # e.g. {'n_estimators': 222, ...}
```

Optuna: Writing the Objective Function

```
def objective(trial):  
    # 1. Suggest hyperparameter values  
    n_estimators = trial.suggest_int('n_estimators', 50, 500)  
    max_depth = trial.suggest_int('max_depth', 2, 20)  
    min_samples = trial.suggest_int('min_samples_split', 2, 20)  
  
    # 2. Build the model with suggested values  
    clf = RandomForestClassifier(  
        n_estimators=n_estimators,  
        max_depth=max_depth,  
        min_samples_split=min_samples,  
        random_state=42)  
  
    # 3. Cross-validate — NEVER on the test set!  
    return cross_val_score(  
        clf, X_train, y_train,  
        cv=5, scoring='roc_auc').mean()
```

Key Rules

`trial.suggest_int()`

Suggest integer. Use `suggest_float()` for continuous values.

`cross_val_score()`

Evaluate on training folds — test set stays sealed.

Return the score

Optuna maximizes/minimizes whatever float you return.

`n_trials=50`

Start with 50; scale to 200+ for production searches.

`study.best_params`

After optimization, retrieve the winning configuration.

Optuna Visualizations — Reading the Results

■ Optimization History

```
plot_optimization_history(study)
```

Best score vs. trial number. Expect a rapid rise, then plateau.

■ Param Importances

```
plot_param_importances(study)
```

Ranks each hyperparameter by its impact on the objective score.

■ Parallel Coordinates

```
plot_parallel_coordinate(study)
```

Each line = one trial. Bright lines = high-scoring runs. Spot winning regions.

■ Contour Plot

```
plot_contour(study, params=['max_depth', 'n_estimators'])
```

2D heatmap for any pair of params — shows sweet spots and interactions.

MLflow — Your ML Experiment Journal

Log Optuna's best results to MLflow:

```
mlflow.set_experiment('Bootcamp_Optuna_Experiments')

with mlflow.start_run(run_name='Best_RF_Model'):
    # 1. Log best hyperparameters
    mlflow.log_params(study.best_params)

    # 2. Train on full training set
    best_model = RandomForestClassifier(
        **study.best_params, random_state=42)
    best_model.fit(X_train, y_train)

    # 3. Log the final metric
    mlflow.log_metric('test_auc', final_test_auc)

    # 4. Save model artifact (.pkl auto-saved!)
    mlflow.sklearn.log_model(best_model, 'rf_model')
```



Log Parameters

Every hyperparameter stored per run



Log Metrics

AUC, accuracy, loss — compare runs side-by-side



Log Artifacts

Saves the actual .pkl model file automatically



Model Registry

`mlflow ui` → <http://localhost:5000>

MLflow UI — What You Can Do



Compare Runs

View all experiments side-by-side. Sort by any metric in seconds.



Filter by Metric

Filter runs where `test_auc > 0.95` to focus on top performers.



Download Artifacts

Download logged .pkl models, charts, or any saved file.



Model Registry

Promote the best model: None → Staging → Production → Archived.



Param Charts

Visualize how each hyperparameter affects the final score.



Reproduce Any Run

Click a run ID to re-run it — all params are stored.

Best Practices — Do's & Don'ts

■ Never tune on the test set

Use cross-validation on training data. Test set = final evaluation only.

■ Log everything to MLflow

You WILL forget your best config. If it's not logged, it doesn't exist.

■ Set n_trials based on your budget

Start 20-50 trials for quick experiments. Scale to 200+ for production.

■ Don't guess hyperparameters

Manual tuning is inefficient. Even random search beats human intuition.

■ Don't skip experiment tracking

Without tracking you can't reproduce results or explain your model to stakeholders.

■ Use nested cross-validation

Prevents overfitting to validation set — gives an unbiased performance estimate.

Key Takeaways

- **Don't Guess — Optimize** Use Optuna to systematically search hyperparameter space. It gets smarter with every trial.
- **Bayesian > Grid > Random** Bayesian Optimization learns from past trials and focuses on the most promising regions.
- ■ **Never Tune on the Test Set** Use cross-validation to guide optimization. The test set is for the final performance report only.
- **Track Everything with MLflow** Experiment tracking is to ML what version control is to software — always use it.
- **MLflow Model Registry** Formalizes the path from experiment to staging to production with a full audit trail.

Quick Reference Quiz

Check your understanding before the next session

Q1 Is `learning_rate` a model parameter or a hyperparameter? Why?

Q2 Why does Bayesian Optimization (Optuna) outperform Grid Search in most real scenarios?

Q3 What is 'overfitting the validation set' and how does nested cross-validation prevent it?

Q4 What terminal command launches the MLflow dashboard? What URL do you open?

Q5 Name 3 things you should ALWAYS log during a model training experiment.